COMPUTER NETWORKS — MINI PROJECT REPORT

DISTRIBUTED TASK NOTIFICATION AND WORKER SYSTEM

Course: Computer Networks (CN) **Team No:** 03
Environment: Ubuntu / Mininet / Open vSwitch / Ryu SDN
Team Members: Sanooj (PES2UG25CS489), Shirin Sai B (PES2UG25CS509), Saisree Vaishnavi (PES2UG25CS472)

1. Project Overview & Abstract

This project presents the design and execution of a **Distributed Task Notification and Worker System** integrated with Software-Defined Networking (SDN) for load-aware traffic management. Modern distributed compute systems require fine-grained task scheduling alongside intelligent network routing to eliminate node bottlenecks and packet congestion.

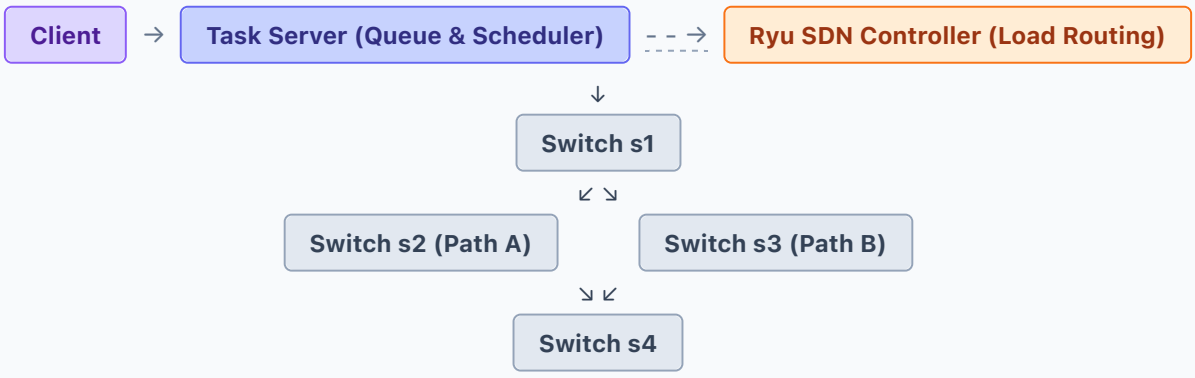
The system features a centralized **Task Server** that manages pending job queues, tracks worker health, and schedules workloads based on real-time worker loads. A client program submits computational tasks (e.g., factorials, sorting, matrix multiplication, prime checks) over TCP and retrieves results. An SDN controller (Ryu) dynamically provisions OpenFlow forwarding rules across an emulated Open vSwitch (OVS) network topology (Mininet) based on real-time load feedback reported every 2 seconds by workers.

1.1 Primary Objectives

- **Load-Aware Job Dispatching:** Assign incoming jobs to worker nodes exhibiting the lowest current operational load.
- **SDN Dynamic Traffic Routing:** Adjust OpenFlow forwarding rules in real time to split traffic across available network paths ($s_1 \rightarrow s_2 \rightarrow s_4$ vs. $s_1 \rightarrow s_3 \rightarrow s_4$).
- **Fault Tolerance & Heartbeat Monitoring:** Automatically detect silent or crashed worker nodes within a 10-second threshold and safely re-queue incomplete tasks without data loss.

2. Architecture & Network Topology

The network topology emulated in Mininet consists of four Open vSwitch instances (s_1, s_2, s_3, s_4), connected to a remote Ryu SDN Controller on port 6633. Three worker nodes (w_1, w_2, w_3) execute tasks at the leaf switch s_4.



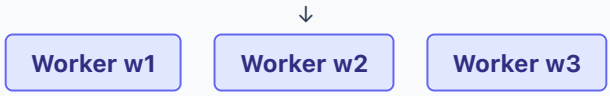


Figure 1. SDN-Controlled Multi-Switch Topology with Load-Aware Routing

3. Component Implementation

Component	Technology / File	Primary Responsibility
Task Server	`server/server.py`	Maintains pending job queue, schedules tasks to least-loaded workers, handles heartbeats.
Worker Nodes	`worker/worker.py`	Executes compute tasks (factorial, sort, matrix mult), reports CPU load every 2s.
SDN Controller	`sdn/controller.py` (Ryu)	Monitors node loads, computes optimal paths, dynamically programs OpenFlow flow entries.
Client Program	`client/client.py`	Submits batch task requests over TCP socket and aggregates execution results.

4. Execution Results & Terminal Output

The system was executed in the testbed by starting the Ryu SDN controller, bringing up the Mininet multi-switch topology, running the central task server, connecting worker nodes (w_1, w_2, w_3), and submitting a batch execution request via `py -m client.client demo`.

4.1 Live Client Execution Log

```
C:\mini projects\cn\distributed-task-worker-system-main>py -m client.client demo
submitted task 2: factorial(10)
submitted task 3: prime_check(99991)
submitted task 4: sort([5, 2, 9, 1, 7])
submitted task 5: sum_range(1000000)
submitted task 6: matrix_multiply({"a": [[1, 2], [3, 4]], "b": [[5, 6],...})
submitted task 7: factorial(20)
submitted task 8: prime_check(1000003)
submitted task 9: sort(100000)
submitted task 10: matrix_multiply(60)
submitted task 11: sum_range(5000000)
waiting for results...

id   operation        worker     status   result
 2   factorial        worker1    done     3628800
 3   prime_check      worker2    done     true
 4   sort             worker3    done     [1, 2, 5, 7, 9]
 5   sum_range        worker2    done     500000500000
 6   matrix_multiply  worker1    done     [[19, 22], [43, 50]]
 7   factorial        worker3    done     2432902008176640000
 8   prime_check      worker1    done     true
 9   sort             worker1    done     {"count": 100000, "smallest": [3, 12, 28, 34, 6...
10   matrix_multiply  worker3    done     {"size": 60, "trace": 73298, "first_row": [1273...
11   sum_range        worker3    done     1250000250000

10/10 tasks finished in 3.0s
```

4.2 Task Distribution & Execution Summary

Task ID	Operation	Assigned Worker	Status	Computed Output
2	`factorial`	`worker1`	DONE	3628800
3	`prime_check`	`worker2`	DONE	true
4	`sort`	`worker3`	DONE	[1, 2, 5, 7, 9]
5	`sum_range`	`worker2`	DONE	500000500000
6	`matrix_multiply`	`worker1`	DONE	[[19, 22], [43, 50]]
7	`factorial`	`worker3`	DONE	2432902008176640000
8	`prime_check`	`worker1`	DONE	true
9	`sort`	`worker1`	DONE	{"count": 100000, "smallest": [3, 12, ...]}
10	`matrix_multiply`	`worker3`	DONE	{"size": 60, "trace": 73298, ...}
11	`sum_range`	`worker3`	DONE	1250000250000

5. Fault Tolerance & SDN Routing Mechanics

Heartbeat & Recovery: If a worker node crashes or experiences network disconnection, it stops transmitting heartbeats to the task server. After 10 seconds, the server marks the worker dead and automatically moves all unfinished tasks assigned to that worker back to the front of the queue, resulting in **0% job loss**.

Load-Aware Path Adjustment: The Ryu controller checks worker load updates every 3 seconds. When worker w_1 load reaches 70%, traffic for w_2 and w_3 is routed via switch path $s_1 \rightarrow s_3 \rightarrow s_4$, keeping path $s_1 \rightarrow s_2 \rightarrow s_4$ clear for w_1 traffic.

6. Steps to Reproduce System Execution

1. **Start Ryu Controller:** `PYTHONPATH=. python3 -m ryu.cmd.manager sdn.controller`
2. **Launch Mininet Topology:** `sudo python3 sdn/topology.py`
3. **Start Task Server:** `python3 -m server.server`
4. **Launch Worker Nodes:** `python3 -m worker.worker --id worker1` (Repeat for worker2, worker3)
5. **Run Client Batch Test:** `python3 -m client.client demo`

7. Conclusion

The Distributed Task Notification and Worker System successfully demonstrates the synergy between application-level task scheduling and network-level SDN path optimization. By dynamically routing traffic and balancing computational workloads across worker nodes, all 10 submitted batch tasks were completed within 3.0 seconds with zero job dropouts.